

叠衣服复现demo

 本指南用于复现电视机叠衣服 Demo 工站，包含环境搭建、数据下载、运行步骤及硬件清单。

一、工站方案与硬件安装

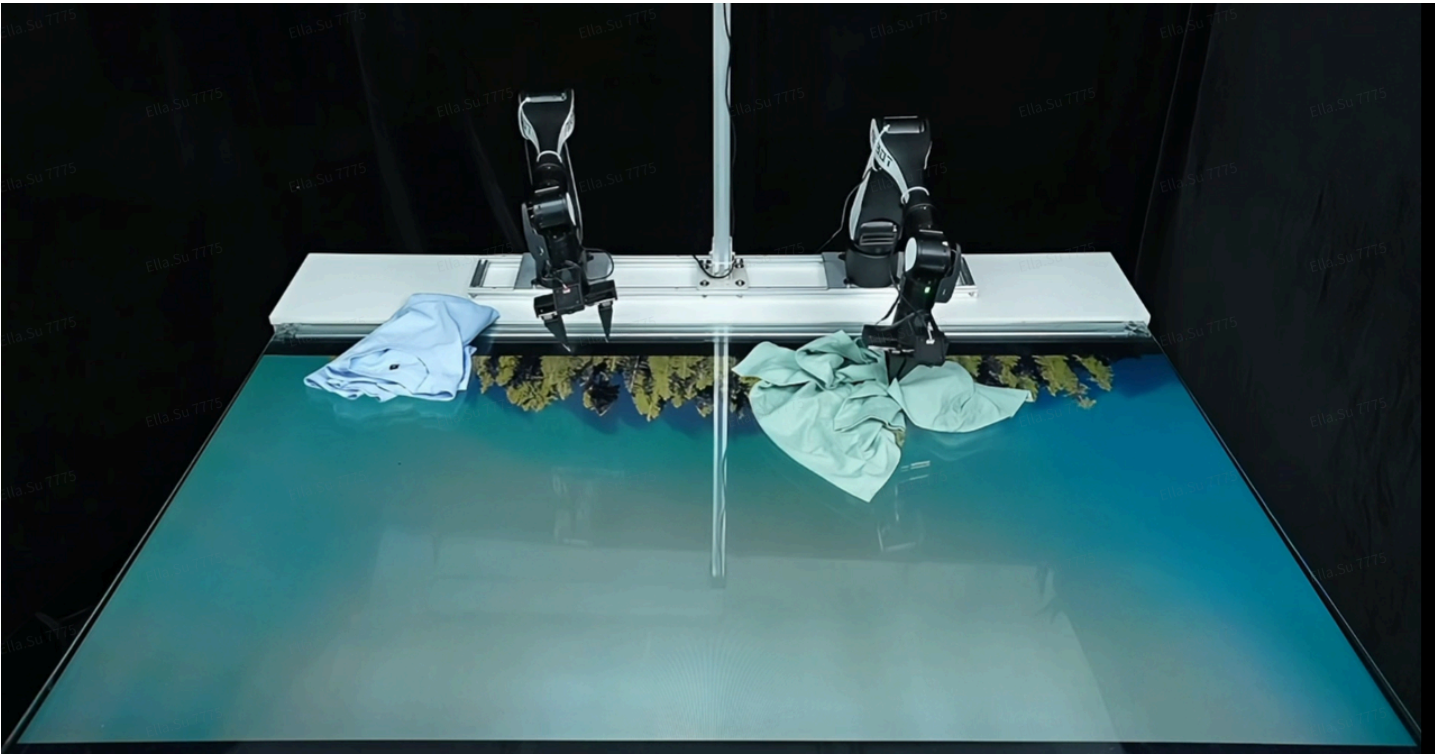
请根据演示目标和当地采购条件选择一种工站方案。完成工作区域、机械臂和相机安装，工作站搭建完成后，再开始软件配置。两种方案使用相同的机械臂、相机和模型，主要区别是台面结构以及是否使用电视画面展示视觉抗干扰能力。

1.1 两种工站方案

方案	适用场景	主要特点	采购与搭建
方案 A：电视机抗干扰版	完整搭建电视机抗干扰演示环境；需要展示模型在动态、彩色背景下的适应能力。	铝型材/板材拼装台架，台面下放置电视机，电视机上方覆盖透明玻璃板。	成本和装配难度较高，需准备电视机、玻璃板、支架模型及安装附件。
方案 B：白色桌面简易版	常规演示、快速部署、海外部署或不要求展示电视背景抗干扰能力。	使用纯白色桌面和玻璃板作为操作区域，不安装台下电视机。	成本较低，采购和搭建更方便，桌子长度不小于 1.7 m，宽度不小于 1.2 m。

注：两种方案用到的衣服都要求是纯色M/L码短袖T恤。

1.2 方案A：电视机抗干扰版



电视机抗干扰版工站整体效果

该方案使用木板、铝型材及配套连接件拼装支撑台架，在操作台面下方水平放置电视机，并在电视机上方覆盖透明玻璃板。电视机播放动态或彩色内容时，机械臂在玻璃板上完成叠衣，用于展示模型对复杂视觉背景的适应能力。

模块	要求 / 说明
台架	按照 B 叠衣支架模型.zip 、 B 叠衣支架0904.zip 和 P 叠衣台架安装说明.pptx 准备木板、铝型材、角码、连接板和紧固件。
电视机	参考型号为小米电视机 S MiniLED 75 英寸低反屏 1008 高分区；也可使用尺寸和显示效果相近的设备。（购买链接参考： https://detail.tmall.com/item.htm?fpChannel=101&fpChannelSig=10b1c2d30edfc2939ea5ecf763cc98e48da870a0&id=1039475756408&skuId=6056687625442 ）
玻璃板	覆盖电视机显示区域并作为衣物操作表面，尺寸为1650x1060x8 mm。
机械臂与相机支架	按照配套支架模型和安装 SOP 固定，确保机械臂底座、腕部相机和环境相机的相对位置稳定。安装方式以 P 叠衣支架安装说明.pptx 为准。

1.3 方案B：白色桌面简易版

该方案使用纯白色桌面和玻璃板作为衣物操作面，不安装台下电视机。桌子高度不限；机械臂数量、角色、相机数量以及相对安装关系与方案 A 保持一致。

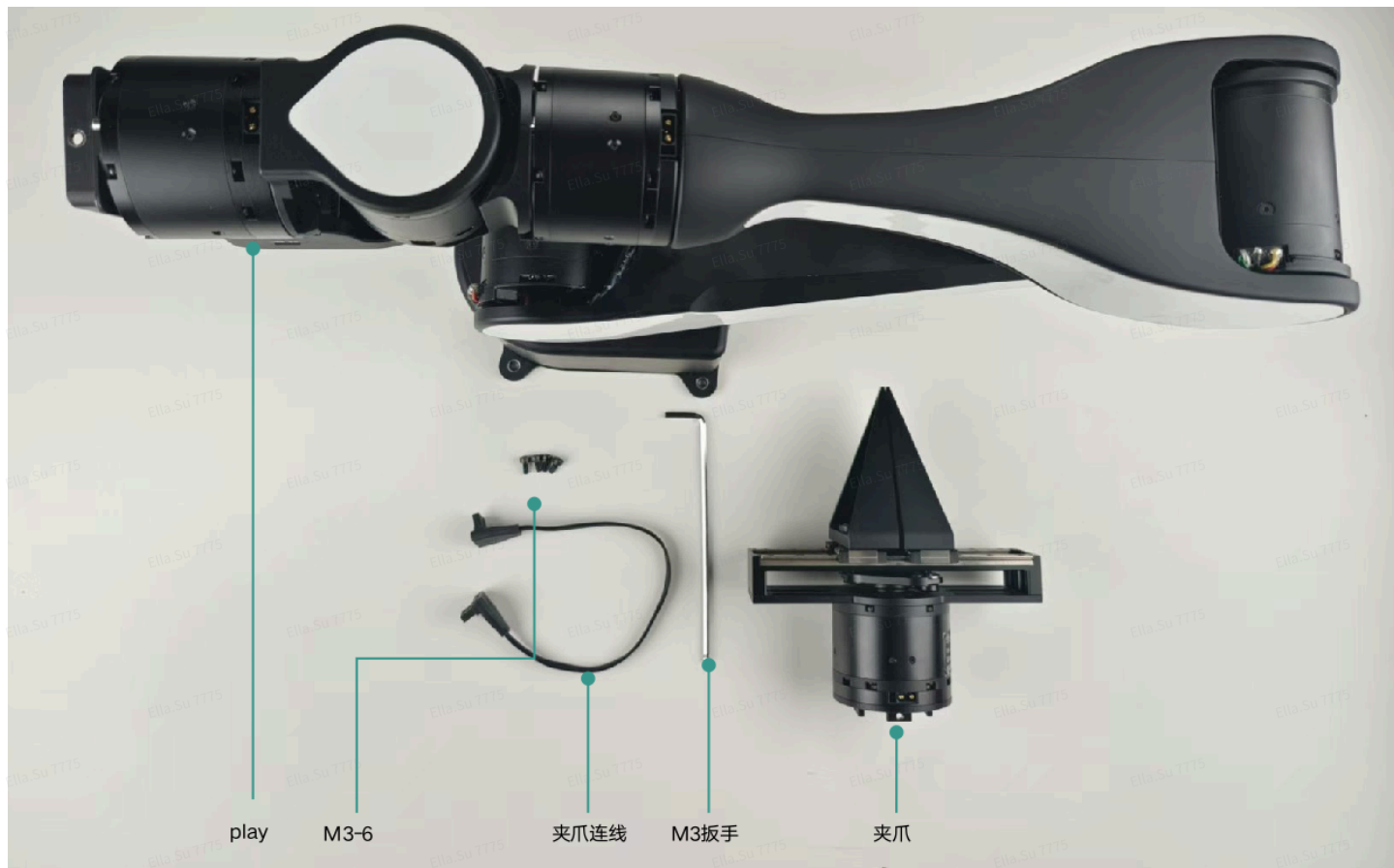
模块	要求 / 说明

桌面颜色及尺寸	纯白色桌面，尺寸要求：长度不小于 1.7 m，宽度不小于 1.2 m；高度不限。（购买链接参考： https://item.taobao.com/item.htm?ali_refid=a3_430673_1006%3A1679766201%3AH%3ALIRpPwYnnLVh4LgSUu40Q%2BEuvhplRnn57DzDuuWucnl%3D%3A4de397306bd03ac2da2ac8159f78b3cc&ali_trackid=282_4de397306bd03ac2da2ac8159f78b3cc&id=731805103039&loginBonus=1&mi_id=0000yk4FxWL4U2SzQFtnHqPaSE8QAM7LtmL7tyAM067_WKQ&mm_sceneid=1_0_3642252468_0&priceTId=2150463a17865148559703395e1208&spm=a21n57.sem.item.34&utparam=%7B%22aplus_abtest%22%3A%2271a1c0cc93a1c030fb780de008cda22a%22%7D&xxc=ad_ztc)
机械臂与相机支架	按照配套支架模型 📎 叠衣支架0904.zip 和安装 SOP 固定，确保机械臂底座、腕部相机和环境相机的相对位置稳定。安装方式以 📎 叠衣支架安装说明.pptx 为准。
玻璃板	覆盖桌面尺寸并作为衣物操作表面，厚度6-8mm。

1.4 方案所需硬件

类型	规格 / 说明
AIRBOT Play 双臂 + 夹爪	两台机械臂，两个夹爪，以及配套的数据线、夹爪线和电源
腕部相机 × 2	720P / 2.3 mm / 75° / 无畸变
环境相机 × 1	1080P / 2.7 mm / 120° / 无畸变

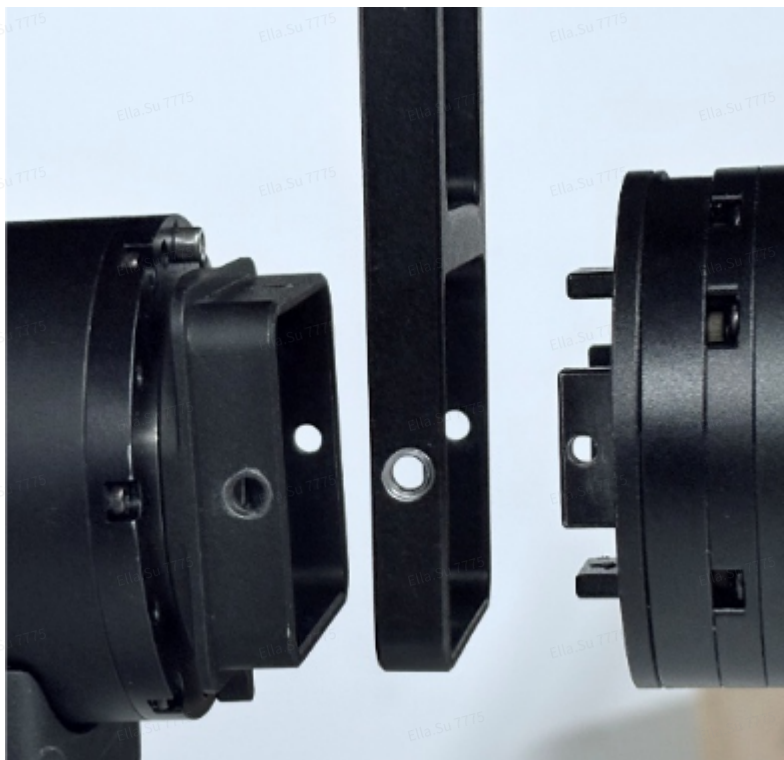
1.5 机械臂安装说明



1. 先将标志向下摆放，如下图所示：



2. 按照下图所示安装相机及其支架（臂-相机-夹爪）、数据线、夹爪线。



1.6 硬件安装位置参考图

以下图片展示台架整体安装效果、环境相机的安装位置。具体固定方式以配套支架模型和安装 SOP 为准。

1.6.1 台架整体安装效果



1.6.2 环境相机



环境相机安装位置（侧面）





二、项目仓库与版本基线

代码仓库地址：https://github.com/Robot-K/Openpi_RL/

2.1 系统要求

- 操作系统：Ubuntu $\geq$ 22.04；
- Python：
 - 机械臂服务：conda Python 3.10；
 - 模型服务：uv Python 3.11；
- GPU：NVIDIA RTX 4090 24GB 或 RTX 5090 32GB

2.2 安装基础工具与uv

```
1 sudo apt update
2 sudo apt install -y git git-lfs curl wget ca-certificates build-essential
  cmake unzip
3 sudo apt install -y iproute2 udev kmod usbutils libcanberra-gtk-module
  libcanberra-gtk3-module
4 git lfs install
5
6 curl --proto '=https' --tlsv1.2 -LsSf https://astral.sh/uv/install.sh | sh
7 export PATH="$HOME/.local/bin:$PATH"
8
9 #检查环境安装状态，均能正常输出则安装成功
10 uv --version
11 git --version
12 lsusb --version | head -n 1
13 nvidia-smi
```

2.3 下载仓库

代码块

```
1 mkdir -p "$HOME/tv_fold_demo"
2 cd "$HOME/tv_fold_demo"
3
4 git clone https://github.com/Robot-K/Openpi_RL.git
```

三、数据与模型下载

3.1 资源地址

资源	Hugging Face 仓库
数据集（mcap）	https://huggingface.co/datasets/xiaoleezuishuai/airbot-fold-cloth-mcap
模型权重	https://huggingface.co/xiaoleezuishuai/policy-v3-wospatiodelta-iter4-tv2-280000

3.2 下载数据集【按需】

下载 mcap 数据集

```
1 uvx --from 'huggingface_hub>=1.0' \
2 hf download \
```

```

3 xiaoleezuishuai/airbot-fold-cloth-mcap \
4 --repo-type dataset \
5 --include 'fold_cloth_v3/*' \
6 --include 'fold_cloth_v3_dagger/*' \
7 --include 'fold_cloth_v4/*' \
8 --include 'fold_cloth_v4_dagger/*' \
9 --local-dir "$HOME/tv_fold_demo/mcap_data"
10
11 ls "$HOME/tv_fold_demo/mcap_data"

```

3.3 下载模型权重

下载模型权重

```

1 #网络波动可能会导致模型下载中断。如中断，删除未完成的缓存目录后重试。
2 uvx --from 'huggingface_hub>=1.0' hf download xiaoleezuishuai/policy-v3-
  wospatiodelta-iter4-tv2-280000 --local-dir
  "$HOME/tv_fold_demo/policy_v3_wospatiodelta_iter4_tv2"
3 ls "$HOME/tv_fold_demo/policy_v3_wospatiodelta_iter4_tv2"

```

四、环境搭建与运行步骤

4.1 创建conda环境

如果系统已经安装并能正常使用 conda，可跳过 Miniconda 安装，直接创建 airdc 环境。未安装 conda 时执行下面的安装命令。

代码块

```

1 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O
  "$HOME/miniconda.sh"
2 bash "$HOME/miniconda.sh" -b -p "$HOME/miniconda3"
3 source "$HOME/miniconda3/etc/profile.d/conda.sh"
4 conda init bash
5 conda create -n airdc python=3.10 -y
6 conda activate airdc
7 python --version

```

4.2 安装 AIRBOT SDK、AIRDC 与推理客户端依赖

4.2.1 安装机械臂软件环境



airbot-configure_5.1.6-1_all.deb

3.38KB



airbot_py-5.1.6-py3-none-any.whl

51.42KB



1. 安装docker

代码块

```
1 sudo apt-get update
2 sudo apt-get install -y ca-certificates curl
3 sudo install -m 0755 -d /etc/apt/keyrings
4 sudo curl -fsSL https://mirrors.ustc.edu.cn/docker-ce/linux/ubuntu/gpg -o
  /etc/apt/keyrings/docker.asc
5 sudo chmod a+r /etc/apt/keyrings/docker.asc
6 echo "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.asc] https://mirrors.ustc.edu.cn/docker-
  ce/linux/ubuntu $(. /etc/os-release && echo
  "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | sudo tee
  /etc/apt/sources.list.d/docker.list > /dev/null
7 sudo apt-get update
8 sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-
  plugin docker-compose-plugin
```

执行完上面的用户组命令后，重启 Ubuntu。重新登录系统后执行：

代码块

```
1 docker ps
```

2. 安装AIRBOT Play驱动

代码块

```
1 sudo apt-get install ./airbot-configure_5.1.6-1_all.deb
2 dpkg -l | grep airbot
```

3. 在 airdc 环境中安装 Python SDK

代码块

```
1 conda activate airdc
```

```
2 python -m pip install -i https://pypi.tuna.tsinghua.edu.cn/simple ./airbot_py-5.1.6-py3-none-any.whl
3 python -m pip list | grep airbot
```

4.2.2 安装 data-collection



data-collection.zip
4.85MB



代码块

```
1 cd data-collection
2 conda activate airdc
3 bash install/install.sh
```

4.2.3 机械臂设备号确认

分别插入左臂和右臂，记录下左右臂对应的设备识别号

代码块

```
1 ip link | grep can
```

4.3 相机识别与画面检查

连接环境相机、左腕相机和右腕相机。相机设备号会随 USB 端口变化，需按照实际情况替换设备号。

代码块

```
1 #查看相机设备号
2 v4l2-ctl --list-cameras
3
4 #检查相机画面是否正常
5 conda activate airdc
6 cd data-collection
7 # 下面的 4 6 8 仅为示例，必须替换为上一步识别出的实际设备号
8 python3 scripts/multi_capture.py 4 6 8 -ff MJPEG MJPEG MJPEG
```

修改配置文件（关键步骤，决定实际使用的相机） `robot_config.py`：

代码块

```
1 cd Openpi_RL/examples/airbot
```

```
2 # 将camera_index修改为实际相机设备号,
3 # 依次为: 环境相机、左腕相机、右腕相机【顺序不要写反】
4 camera_index: list[int | str] = [4, 6, 8]
```

```
from typing import Any
from pydantic import BaseModel

class RobotConfig(BaseModel):
    """Configuration for the AIRBOT robot."""

    robot_type: str = "play"
    robot_groups: list[str] = ["left", "right"]
    robot_ports: list[int] = [50051, 50053]
    leader_ports: list[int] = [50050, 50052]
    camera_names: list[str] = ["base_0_rgb", "left_wrist_0_rgb", "right_wrist_0_rgb"]
    camera_index: list[int | str] = [4, 8, 6]
    camera_pixel_format: list[str] = ["MJPEG", "MJPEG", "MJPEG"]
    camera_width: list[int] = [640, 640, 640]
    camera_height: list[int] = [480, 480, 480]
    camera_fps: list[int] = [30, 30, 30]

    def model_post_init(self, __context: Any) -> None:
        for _ in range(len(self.camera_names) - len(self.camera_index)):
            self.camera_names.pop()
        while len(self.camera_pixel_format) < len(self.camera_names):
            self.camera_pixel_format.append("YUYV")
        if len(self.camera_pixel_format) > len(self.camera_names):
            self.camera_pixel_format = self.camera_pixel_format[: len(self.camera_names)]
        for attr, default in (("camera_width", 640), ("camera_height", 480), ("camera_fps", 30)):
            values = getattr(self, attr)
            while len(values) < len(self.camera_names):
                values.append(default)
            if len(values) > len(self.camera_names):
                setattr(self, attr, values[: len(self.camera_names)])
```

五、配置模型服务与推理脚本

编辑 \$HOME/tv_fold_demo/Openpi_RL/scripts/cmds/serve_policy.sh, 将 CONFIG 区设为:

代码块

```
1 cd Openpi_RL/examples/airbot/cmds
2 CHECKPOINT_DIR="${HOME}/tv_fold_demo/policy_v3_wospatiodelta_iter4_tv2"
3 PORT=8000
```

六、启动DEMO

6.1 启动机械臂

代码块

```
1 左臂: airbot_fsm -i can0 -p 50051
```

```
2 右臂: airbot_fsm -i can1 -p 50053
```

6.2 启动 serve 命令

代码块

```
1 export UV_PYTHON=3.11
2 bash serve_policy.sh
```

6.3 启动推理命令

启动前先修改推理脚本配置:

```
Openpi_RL > scripts > cmds > $ infer_async.sh
```

```
1  #!/usr/bin/env bash
2  # 异步推理 (Async): 推理与执行并行, 支持 TCS 时序 chunk 平滑, 实时性更好。
3  # 推荐用于正式部署。
4  set -euo pipefail
5  cd "$(dirname "$0")/../../.."
6
7  # CONFIG
8  HOST=127.0.0.1
9  PORT=8000
10 PROMPT="Fold clothes"
11
12
13 # TCS (Temporal Chunk Smoothing) 平滑参数:
14 # tcs_min_overlap 新旧 chunk 混合 (blend) 时的最小重叠窗口长度;
15 # 重叠区内按线性权重从旧-新渐变, 消除动作跳变
16 # max_inference_rate 最大推理频率 (Hz), 限制推理请求速率, 避免过度切换动作;
17 # initial_action_wait_s 首帧启动等待时限 (秒): episode 开始时等待第一个
18 # action chunk 到达的最长时间, 超时则保持当前姿态直到推理就绪
19 # tcs_drop_max 推理延迟补偿: 新 chunk 到达时, 丢弃已过期的前 N 步
20 # (N = min(实际延迟步数, tcs_drop_max)), 避免执行过时动作
21
22 MAX_INFERENCE_RATE=2 # 最大推理请求频率 (Hz); 0 = 不限速
23 STEP_RATE=200 # 动作发布频率 (Hz)
24 INITIAL_ACTION_WAIT_S=1.0
25 TCS_MIN_OVERLAP=3
26 TCS_DROP_MAX=12
27
28 INTERPOLATE=true # true = 启用动作插值平滑
29 RECORD=false # true = 保存 MCAP 数据
30 RECORD_DIR=./inference_data/dagger_1550_1560
31 DAGGER=false # true = 启用 Dagger 干预采集 (需要主臂连接)
32 #
```

代码块

```
1 INTERPOLATE=true # 启用动作插值平滑
2 DAGGER=false # 不用示教臂时改为 false, 避免连接 leader 端口报错
```

保存后执行：

代码块

```
1 conda activate airdc
2 bash infer_async.sh
```



操作快捷键： Enter = 开始 | D = 舍弃 | Q = 退出

七、附件

-  [叠衣支架模型.zip](#)
-  [叠衣台架安装说明.pptx](#)
-  [叠衣支架安装说明.pptx](#)
-  [叠衣支架0904.zip](#)